Highlights

A Memento-, Hope-, and Reflection-Enhanced Large Language Model Pipeline for Simulated Joint-Operation Plan Generation and Evaluation

Changrui Zhang, Chengwei Yang

- Repository-grounded elsarticle draft for simulated joint-operation planning
- Full pipeline improves mission success over the pure LLM baseline
- Five-scenario scene-out evaluation shows Full beating Pure in 5/5 cases
- Draft reports only code-supported mechanisms and source-traceable results

A Memento-, Hope-, and Reflection-Enhanced Large Language Model Pipeline for Simulated Joint-Operation Plan Generation and Evaluation

Changrui Zhang[a], Chengwei Yang[a,*]

[a] *Beijing Institute of Technology, Beijing, China*

Abstract

This paper presents an evidence-grounded English manuscript draft for a simulation-oriented planning pipeline built around three implemented components: Memento-style case retrieval, a Hope adaptive weighting controller, and a reflection loop for iterative prompt refinement. The system generates structured joint-operation plans, evaluates them through a five-stage kill-chain simulation model, and exports the selected plan into DoDAF OV-5b, DoDAF OV-6c, and C2SIM artifacts. To avoid unsupported claims, the draft only describes mechanisms that are explicitly present in the current codebase and experiment outputs.

The main single-scenario ablation study uses the repository's sample coastal joint-assault scenario with 10 optimization iterations, 50 Monte Carlo simulation runs, a fixed random seed of 7, and writeback disabled. Under this setting, the full pipeline improves mission success from 0.6260 to 0.6388, overall effectiveness from 0.7328 to 0.7685, and the loss-exchange ratio from 1.6564 to 1.9771 relative to the pure large language model baseline. The main cross-scenario generalization study uses five scenario families and leave-one-source-scenario-out case banks derived from 250 source cases. In that evidence set, the full pipeline outperforms the pure baseline in all five scenarios, increasing average mission success from 0.6595 to 0.6828 and average overall effectiveness from 0.7417 to 0.7924, while the relative ordering among the Memento-only and Hope-only variants remains scenario-dependent.

The draft also documents the remaining submission-oriented limitations,

*Corresponding author.

Email address: yangchengwei@bit.edu.cn (Chengwei Yang)

including residual scenario dependence in module gains and the fact that final journal packaging still depends on separate graphical-abstract and editorial artwork checks. The refreshed canonical result bundles now serialize a top-level runtime manifest that records the local backend identifier, software stack, and available CPU/GPU metadata for the reported evidence.

Keywords: large language models, case-based reasoning, simulation-based evaluation, adaptive weighting, structured planning

---

1. Introduction

Recent transformer and foundation-model advances have made large language models (LLMs) increasingly capable of flexible multi-step planning and decision support, which makes them attractive for simulation-centric scenario exploration workflows [1–8]. Reasoning-time performance can be further shaped by prompt-side strategies such as chain-of-thought, self-consistency, ReAct-style tool interaction, deliberate search over thought branches, and explicit tool use [9–13]. At the same time, planning quality remains sensitive to context selection, prior experience, and iterative self-correction. Repository-local evidence for the current project suggests that three design directions are central to its implementation: retrieval-style memory augmentation, adaptive capability weighting, and reflection-driven prompt adjustment.

The memory side of the project is closer to case-based reasoning (CBR) and retrieval-augmented generation than to standalone prompting. The codebase stores structured cases, retrieves them through a FAISS-backed semantic index when available, and falls back to keyword retrieval otherwise [14–17]. In broader architectural terms, that places it closer to external-memory and memory-network families [18–20] and to classical CBR formulations [21, 22]. The reflection side is closer to iterative self-improvement and agent-memory schemes, including Reflexion, Self-Refine, generative-agent memory, prompt optimization, and the recent Memento line, although the current implementation should still be interpreted strictly as a repository-specific planning loop rather than as a reimplementation of any single external framework [23–27].

Within this repository, the target problem is not free-form text generation. Instead, the system aims to produce structured joint-operation plans that can be simulated, compared under ablation settings, and exported to architecture and interoperability artifacts. The simulation layer scores plans

through a five-stage kill-chain abstraction, computes mission-level metrics such as mission success and overall effectiveness, and aggregates repeated stochastic runs into Monte Carlo summaries. At an operational-reading level, the five stages loosely map to target discovery, disruption and suppression, breach of defended nodes, control of key objectives, and sustainment of tempo and support. This mapping is intended only as a theory-alignment aid: the implemented simulator is still a repository-local operational simplification inspired by joint-targeting, F2T2EA, and network-centric command thinking rather than a doctrinal one-to-one reproduction [28–31]. The export layer produces DoDAF and C2SIM outputs aligned with repository-local structured plan objects and broader simulation-interoperability practice [32–35].

This draft intentionally stays close to what can be verified from files in the current project. It does not claim formal guarantees, external benchmark coverage, or a complete literature survey. Instead, the paper makes four narrow contributions grounded in code and experiment outputs:

1. It documents the repository's planning pipeline as an integrated workflow that combines case retrieval, adaptive weighting, reflection, structured simulation evaluation, and standards-oriented exporters.
2. It reports a single-scenario four-way ablation using the repository's canonical ablation suite.
3. It reports a five-scenario cross-scenario generalization study using the repository's canonical scene-out evaluation suite.
4. It surfaces practical limitations that remain before journal submission, including that the final Elsevier submission package still depends on separate graphical-abstract, grayscale-artwork, and editorial asset checks even after the canonical evidence bundles were refreshed with machine-readable runtime manifests.

The rest of the paper describes the implemented pipeline, the selected evidence scope, the main quantitative findings, and the remaining submission-oriented cleanup items.

2. Method

The repository is organized around structured scenario, plan, simulation, and export objects rather than free-form prompting alone. This section deliberately limits itself to mechanisms that are directly supported by the current code and stored outputs.

2.1. Pipeline overview

Figure 1 presents the overall framework of the implemented planning pipeline. The framework starts from a structured scenario representation and couples two auxiliary mechanisms before each generation step: a Memento memory module that retrieves and filters prior cases, and a Hope adaptive controller that transforms scenario attributes into a bounded capability state. These two information sources, together with optional reflection patches from the previous iteration, are fused during prompt construction for LLM-based structured plan generation. The resulting candidate plan is then evaluated by the simulation layer through the repository's five-stage kill-chain abstraction, which yields mission-level metrics and stage-level deficits. These evaluation signals are used in two feedback paths: one path updates the adaptive capability state, and the other path produces reflection guidance for the next generation round. After the iterative loop terminates, the highest-scoring plan is selected and exported into the downstream structured artifacts used by the repository.

2.2. Structured scenario input and plan representation

The domain layer defines normalized capability dimensions (`fires`, `mobility`, `protection`, `awareness`, `ew`, `sustainment`, and `c2`), typed actions, force units, scenarios, plan phases, and complete combat plans. These objects enforce non-empty text fields, bounded numeric ratios, and structured phase/action serialization. The downstream simulator and exporters therefore operate on typed fields rather than on unconstrained narrative text.

2.3. Memento memory module: case retrieval and selection

The case-memory module stores records with mission type, terrain, objective, roles, key actions, lessons, outcomes, and metric summaries. When optional embedding dependencies are available, the module uses a Sentence-Transformer encoder with a FAISS index; otherwise it falls back to keyword retrieval [14, 16, 17]. At the design level, that retrieval path remains closer to case-based reasoning than to standalone prompting [21, 22]. Retrieved items are scored by confidence, blended with a case-quality heuristic, and labeled as either *imitate* or *avoid* according to the source outcome.

The pipeline does not pass all retrieved cases directly to the generator. Instead, it computes a scenario-dependent support score, sorts candidates

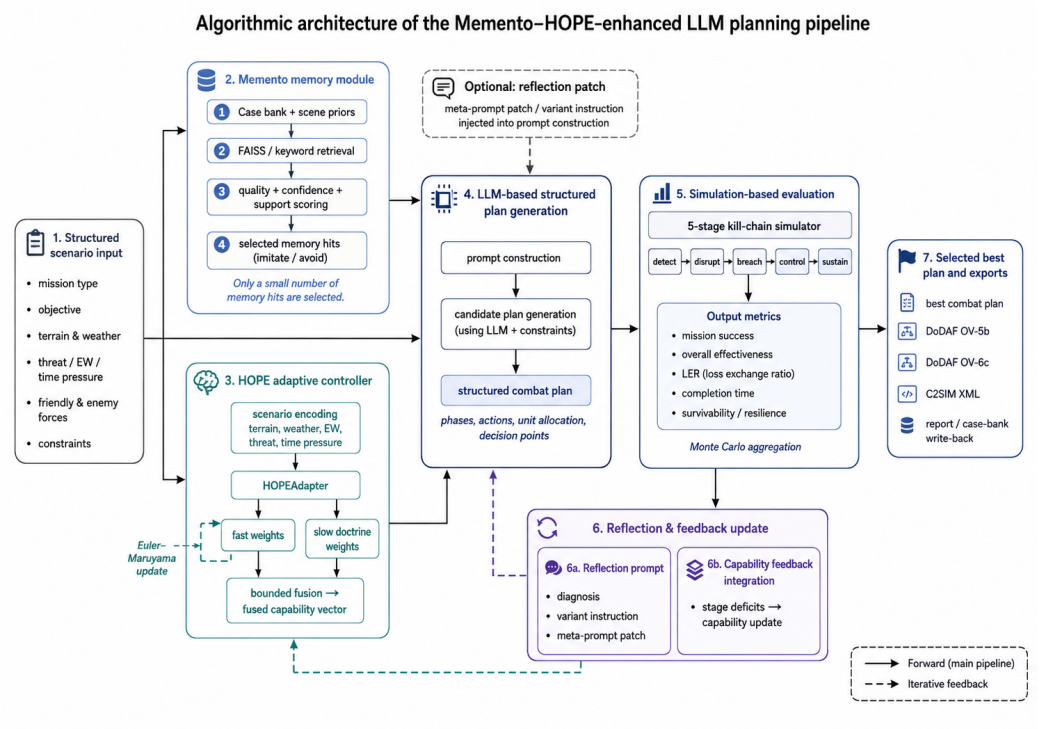

Figure 1: Overall framework of the Memento–HOPE-enhanced LLM planning pipeline. The framework maps structured scenario input to four tightly connected stages: memory retrieval and selection, adaptive capability modulation, LLM-based structured plan generation, and simulation-based evaluation. The solid arrows denote the main forward computation from scenario encoding to best-plan selection and export, whereas the dashed arrows denote the iterative feedback paths that inject reflection guidance and capability updates into subsequent planning rounds.

by support, quality, and confidence, and keeps at most two selected memory hits in strict mode. For sparse generalization settings, the pipeline can also inject hand-authored scene priors for coastal assault, mountain reconnaissance, maritime sustainment, and urban defense scenes. These priors are represented as standard case records and enter the same scoring and selection path.

From a data-lifecycle perspective, the implemented memory path is a four-step loop: offline case curation, line-delimited JSONL persistence, online retrieval and reranking, and optional write-back behind a quality gate. Each stored `CaseRecord` keeps a compact but typed field set. The mission-context

fields are `case_id`, `mission_type`, `terrain`, `objective`, `friendly_roles`, and `enemy_roles`; the action-and-outcome fields are `key_actions`, `lessons`, `outcome`, `tags`, and `metrics`; and the maintenance fields are `update_count`, `last_updated`, and `source_scenarios`. The repository's `_load_records()` routine reads the JSONL bank one line at a time, `retrieve()` chooses the FAISS path when semantic backends are available and otherwise falls back to keyword similarity, and the pipeline-level `_select_memory_hits()` stage reorders candidates by support score, quality, and confidence before truncation. If write-back is enabled, `_write_back_case()` creates a candidate record only after the simulated plan clears mission-success, effectiveness, and minimum-stage thresholds, and `upsert_record()` then merges or appends the result inside the same JSONL store. For the canonical ablation and generalization evidence used in this paper, that final write-back step is disabled in the recorded configurations, so the present manuscript should be read as documenting the memory read-and-use path rather than an online continual-learning experiment.

2.4. HOPE adaptive controller: scenario encoding, fast weights, and bounded fusion

The adaptive controller maintains a slow capability-weight vector, a fast adjustment vector, and a neural adapter named `HOPEAdapter`. The scenario encoder converts terrain and weather into one-hot vectors and appends three scalar features: electronic-warfare threat, threat level, and time pressure. The adapter contains an input gate, a candidate projection, an output projection, and a value estimator that modulates a learned forget gate. In implementation terms, the forget gate depends on an estimated value term and a log-age term, which allows the hidden state to be refreshed during feedback updates while remaining stable during inference. Architecturally, this design borrows the language of gated recurrent updates [36], but it remains a lightweight scenario-conditioned controller rather than a large conditional-computation family such as sparsely gated MoE, GShard, Switch Transformer, GLaM, sparse upcycling, soft MoE, or LLM-to-MoE upcycling systems [37–44].

Fast weights are updated by an Euler–Maruyama-style step:

$$W_{\text{fast}} \leftarrow W_{\text{fast}} + \theta(\hat{W}_{\text{fast}} - W_{\text{fast}})\Delta t + \epsilon\sqrt{\Delta t}\,\xi,$$

where the target vector $\hat{W}_{\text{fast}}$ comes from the adapter, θ is a recovery coefficient, and ϵ controls the noise term. In operational terms for the im-

plemented controller, θ governs how quickly capability emphasis is pulled toward the scenario-conditioned target after feedback, ϵ controls the amount of exploratory perturbation retained under uncertainty, Δt denotes one discrete planning iteration, and ξ is the zero-mean random perturbation used to avoid a rigid fast-weight trajectory. These meanings are best understood as engineering interpretations inside the current simulator rather than as externally calibrated physical units. The resulting fast weights are clipped to scenario-dependent lower and upper bounds. The fused capability vector is then formed by adding a scaled fast component to the slow weights and enforcing mission- and threat-specific floors on key dimensions such as fires, mobility, awareness, and C2. The draft therefore uses standard stochastic-update and nonlinear-control vocabulary to describe the mechanism, but it does not claim a formal stability proof for the implemented controller [45, 46].

2.5. LLM-based structured plan generation

The plan-generation stage receives three classes of conditioning signals from the upstream modules: the structured scenario description, the selected memory hits labeled as *imitate* or *avoid*, and the fused capability state computed by the Hope controller. These elements are injected into prompt construction together with repository-local planning constraints and any optional reflection patch. The LLM is then asked to generate a candidate combat plan in a structured format rather than unconstrained free text.

The resulting plan is normalized into typed phases, actions, unit allocations, and decision points so that downstream evaluation and export can operate on stable fields. In this sense, the LLM stage is not treated as a standalone text generator; it acts as the proposal engine inside a larger structured loop whose inputs and outputs are both repository-defined objects.

2.6. Simulation-based evaluation

The simulator evaluates each plan phase against a five-stage kill-chain model. Stage scores are combined into phase success, mission success, loss-exchange ratio (LER), survivability, command resilience, completion time, and an overall effectiveness score. The detect–disrupt–breach–control–sustain decomposition is best read as a repository-local operational abstraction that is loosely informed by joint-targeting workflows and network-centric command thinking rather than by any single formal doctrinal template [28–30]. More concretely, `detect` corresponds to situation and target discovery, `disrupt` to suppressive interference with enemy action, `breach` to breaking

through defended obstacles or critical nodes, `control` to seizing and maintaining command over key objectives, and `sustain` to preserving tempo, support, and continuity. This mapping is intentionally loose and should be read as a theory-alignment bridge for simulation interpretation, not as a doctrinal one-to-one template. Under repeated stochastic evaluation, the pipeline aggregates multiple simulation runs and stores the mean, standard deviation, minimum, maximum, and 95% confidence interval for selected metrics [31].

2.7. Reflection and capability feedback update

After each planning iteration, the controller derives stage deficits from the simulator's five-stage scores (`detect`, `disrupt`, `breach`, `control`, and `sustain`). These deficits are projected onto capability dimensions through fixed stage-to-capability mappings and an additional bottleneck-boost term. The adapter is then trained with a smooth-L_1 objective toward a bounded desired fast-weight target. Slow weights are updated through a bounded transformation that depends on the utility gradient, a delay-correction term, and minimum floors on selected offensive dimensions. The current code therefore supports adaptive, bounded, feedback-driven weight updates, but this draft does not infer a formally verified optimizer or theorem-backed convergence guarantee from those implementation choices.

Reflection is handled separately from the adaptive weight update. When enabled, the pipeline calls a reflection prompt that summarizes diagnosis items, variant instructions, and a meta-prompt patch for the next planning round. Reflection can be suppressed for some scene types depending on scenario properties and memory support [23, 24].

2.8. Best-plan selection and standards-oriented export

The same selected plan can also be exported into repository-local DoDAF OV-5b, DoDAF OV-6c, and C2SIM representations [32–35]. These exporters are deterministic transformations from structured plan fields to machine-readable JSON or XML artifacts, which helps connect the narrative plan output to downstream modeling and interoperability workflows.

Table 1 gives a representative export example from the selected Full-setting coastal joint-assault result bundle. The key point is that the repository does not export an unrelated post-processed summary: the same structured plan header and phase objects are preserved across the OV-5b activity view, the OV-6c event-trace view, and the C2SIM order representation, with

Table 1: Representative cross-standard export correspondence for the Full-setting coastal joint-assault example under the selected scene-out suite. The table is derived from `best_plan.json`, `dodaf_ov5b.json`, `dodaf_ov6c.json`, and `c2sim.xml` in the same result directory.

Element	Structured plan object	OV-5b	OV-6c	C2SIM
Operation header	Plan title, mission type, commander intent, end state, and control rules.	`operation_name`	`operation_name`	Header / order fields
Phase 1	`phases[0]` (`P1`); actions: `recon`, `c2`.	`ACT-01`	`EVT-01`	`Task P1`
Phase 2	`phases[1]` (`P2`); actions: `strike`, `mobility`, `ew`.	`ACT-02`	`EVT-02`	`Task P2`
Phase 3	`phases[2]` (`P3`); actions: `recon`, `strike`, `mobility`.	`ACT-03`	`EVT-03`	`Task P3`
Phase 4	`phases[3]` (`P4`); actions: `control`, `sustain`.	`ACT-04`	`EVT-04`	`Task P4`
Post-plan feedback	Simulation assessment metrics and writeback gate.	–	`EVT-05`	–

a final OV-6c closure event used to attach simulation feedback to the next planning round.

3. Experimental Setup

This draft uses a single canonical evidence scope to avoid mixing results from incompatible output directories. The goal is to keep the first English manuscript internally consistent and fully traceable to repository-local files.

3.1. Evidence scope and repository constraints

The selected single-scenario ablation evidence comes from the canonical ablation suite (`result/ablation_suite_seed7_v3`). The selected cross-scenario generalization evidence comes from the canonical scene-out suite (`result/generalization_suite_seed7_v2_sceneout_iter20`). The manuscript excludes other result directories from the main body and treats them as audit-only artifacts.

The sample ablation scenario is a coastal-urban assault case with rain, threat level 0.74, electronic-warfare threat 0.68, time pressure 0.77, five friendly units, and three enemy units. Its objective is to seize a coastal landing window and suppress enemy air-defense and shore-defense nodes for follow-on amphibious expansion. The frozen seed case bank used by the ablation suite contains five records.

The generalization study uses five repository-local scenarios listed in `data/generalization_scenarios/manifest.json`: coastal assault, urban defense, mountain reconnaissance, river crossing assault, and maritime

sustainment. The selected evidence set relies on `data/generalization_case_banks_v2_sceneout/summary.json`, which records 250 source cases and five leave-one-source-scenario-out case banks with 200 cases each.

3.2. Ablation configuration

The ablation suite is generated by `run_ablation_suite.ps1` and summarized by `military_research/summarize_ablations.py`. In the canonical evidence set, all four ablation groups use the same random seed (7), the same sample scenario, 10 planning iterations, and 50 Monte Carlo simulation runs, with writeback disabled in the recorded experiment configuration. The four compared settings are:

- **Pure LLM**: memory, Hope, and reflection are disabled;
- **LLM + Memento**: memory is enabled, Hope and reflection are disabled;
- **LLM + Hope**: Hope is enabled, memory and reflection are disabled;
- **Full**: memory, Hope, and reflection are all enabled.

3.3. Cross-scenario generalization configuration

The generalization suite is generated by `run_generalization_suite.ps1`, which calls the ablation runner for each scenario and then summarizes the suite with `military_research/summarize_generalization.py`. In the selected evidence set, each scenario uses 20 planning iterations, 50 Monte Carlo simulation runs, random seed 7, writeback disabled, and a scene-specific leave-one-source-scenario-out case bank.

Table 2 summarizes the five repository-local scenarios used in the selected scene-out suite. The table keeps only the scenario attributes that matter most for experimental reading: mission family, terrain and weather, operational focus, force size, and the three normalized pressure coefficients that feed the planning pipeline.

3.4. Metrics

The paper reports the same metrics stored in `full_result.json`: mission success, overall effectiveness, loss-exchange ratio (LER), completion time in hours, command resilience, stage scores for the five kill-chain stages, and

Table 2: Overview of the five scenario families used in the selected scene-out generalization suite. Threat, EW, and time are the normalized scenario coefficients stored in the repository-local JSON files. Each scene-out case bank contains 200 cases derived from the same 250-case source pool with the target scenario held out.

Scenario	Family	Terrain / weather	Operational focus	Forces	Thr.	EW	Time
Coastal joint assault	assault	coastal-urban / rain	Landing-window seizure; suppress shore and air defense for follow-on expansion.	5 / 3	0.74	0.68	0.77
Urban hub defense	defense	urban / fog	Hold the transport and communications hub against an armored urban thrust.	4 / 2	0.69	0.54	0.66
Mountain corridor reconnaissance	recon	mountain / cloudy	Gather fire-node and supply-line intelligence under constrained visibility.	3 / 2	0.58	0.46	0.55
River crossing breakthrough	assault	river-plain / overcast	Open a defended crossing window and secure the bridgehead.	4 / 2	0.71	0.57	0.73
Island resupply corridor	sustain	maritime-island / windy	Keep the island resupply corridor open under harassment and interdiction.	4 / 2	0.63	0.59	0.61

Monte Carlo confidence intervals where available. Table 3 summarizes the five headline metrics used throughout the main result tables and figures.

In implementation terms, the current code computes mission success from a weighted combination of stage-level and phase-level success, while overall effectiveness combines mission success, LER, survivability, command resilience, and a time penalty. For the Monte Carlo summaries, the reported headline values in the manuscript are arithmetic means over the repeated simulation runs, and confidence intervals are reported where the stored result object provides them.

3.5. Runtime metadata

The refreshed canonical manuscript evidence was rerun with the local backend model `deepseek-r1-0528-qwen3-8b` inside the Python virtual environment `Memento`. The recorded runtime manifests report Python 3.12.11, PyTorch 2.2.0+cu121, the OpenAI-compatible local endpoint at `http://localhost:1234/v1`, a 13th Gen Intel(R) Core(TM) i7-13700F CPU, and an NVIDIA GeForce RTX 3060 GPU with CUDA available. Newly generated `full_result.json` files now emit this information through a top-level `runtime_manifest` so that the main paper evidence can be traced to machine-readable backend, software, and hardware metadata without

Table 3: Definitions of the main quantitative metrics reported in the manuscript. The wording follows the repository's simulator implementation rather than an external benchmark specification.

Metric	Repository-local meaning and code basis	Direction	Stored field
Mission success	Mission-level success score. Computed from a weighted combination of stage-based success across the five kill-chain stages and average phase success, with extra penalties when C2 is weak or time pressure is extreme.	Higher	`mission_success`
Overall effectiveness	Composite utility score for overall plan quality. Combines mission success, normalized LER, survivability, command resilience, and a bounded completion-time term into one summary score.	Higher	`overall_effectiveness`
Loss-exchange ratio (LER)	Enemy-loss to friendly-loss ratio under the simulator's phase-by-phase loss accounting. Values above 1 indicate that the plan inflicts more loss on the opposing side than it absorbs.	Higher	`ler`
Command resilience	Proxy for the plan's ability to maintain command continuity under pressure. Computed from the fused C2, EW, and awareness weights, with a subtraction term for scenario EW threat.	Higher	`command_resilience`
Completion time	Total simulated execution time of the plan in hours. This value also enters the overall-effectiveness score through a bounded time term, so slower plans are penalized both as a standalone metric and inside the composite score.	Lower	`completion_time_hours`

storing machine-specific secrets.

3.6. Reproducibility gaps

Two gaps remain important for a submission-ready version. First, the primary ablation and 20-iteration scene-out evidence now carry machine-readable runtime manifests, but any manuscript interpretation still depends on the repository's own simulator and should not be read as external validation. Second, the current PDF can be compiled with the workspace-local TinyTeX toolchain, but the final submission package still needs the remaining editorial cleanup, including any final grayscale artwork verification and journal-specific packaging checks. These missing pieces are documented explicitly in the manuscript notes rather than being guessed.

Table 4: Main ablation results on the sample coastal joint-assault scenario. All groups use 10 planning iterations, 50 Monte Carlo runs, seed 7, and writeback disabled.

Setting	Mission success	Overall effectiveness	LER	Time (h)	Command resilience
Pure LLM	0.6260	0.7328	1.6564	11.41	0.6096
LLM + Memento	0.6271	0.7221	1.6301	14.59	0.6096
LLM + Hope	0.6229	0.7572	1.7550	9.93	0.7156
Full	0.6388	0.7685	1.9771	14.14	0.6852

Table 5: Five-stage kill-chain scores for the ablation study.

Stage	Pure LLM	LLM + Memento	LLM + Hope	Full
detect	0.6227	0.6263	0.6357	0.6303
disrupt	0.5959	0.6349	0.5954	0.6038
breach	0.6020	0.5766	0.5790	0.5890
control	0.6294	0.6307	0.6076	0.6590
sustain	0.6291	0.6505	0.6187	0.6874

4. Results

4.1. Single-scenario ablation

Table 4 reports the main ablation metrics for the canonical single-scenario evidence set. All values in this subsection are copied from the corresponding `full_result.json` files in the canonical ablation suite. Relative to the pure LLM baseline, the full pipeline improves mission success by 0.0128, overall effectiveness by 0.0357, and LER by 0.3207. The Hope-only variant already provides the strongest isolated gain in overall effectiveness and LER, whereas the memory-only variant only marginally raises mission success and instead degrades the composite score because it stretches completion time substantially.

The stage-level scores in Table 5 help explain where the improvement is concentrated. In the refreshed evidence set, the full system is strongest on `sustain` and `control`, while `breach` remains its weakest stage. The resulting pattern is more uneven than in the earlier draft: the clearest recovery appears in `sustain` and `control`, whereas `detect` and `disrupt` stay close to baseline and `breach` remains the most stubborn weak point.

Figure 2 presents the same five-stage evidence in a more comparison-oriented form. Two patterns are visually stable across the refreshed ablation. First, `breach` remains the lowest-scoring stage of the Full setting and also

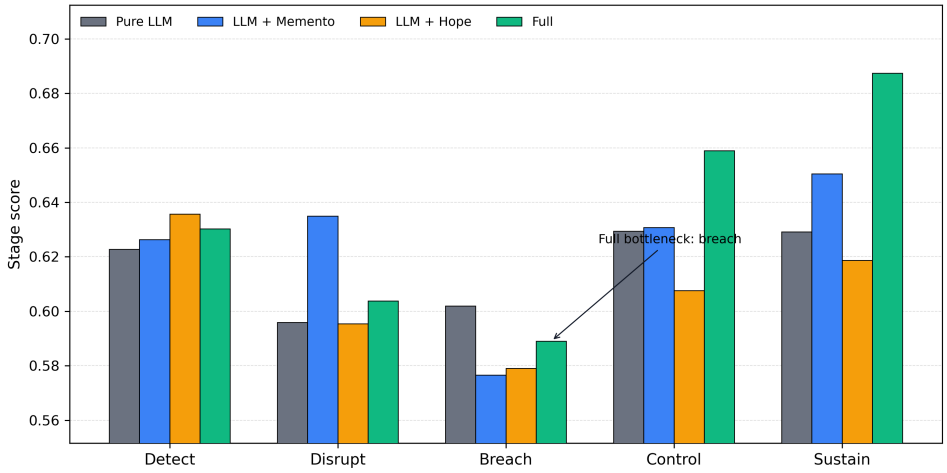

Figure 2: Five-stage kill-chain scores for the canonical single-scenario ablation. The figure visualizes the same values reported in Table 5. In the refreshed evidence, the Full pipeline shows its clearest relative gains in `control` and `sustain`, while `breach` remains the weakest Full-stage score and therefore the most explicit residual bottleneck.

slips slightly below the pure baseline, which makes it the clearest residual bottleneck in this sample scenario. Second, the full pipeline does not improve all five stages uniformly: the clearest recovery appears in `control` and especially `sustain`, whereas `detect` and `disrupt` remain close to baseline.

4.2. Cross-scenario generalization

The selected generalization evidence covers five scenario families. Table 6 reports the suite-level aggregate outcome first. In this refreshed set, the full pipeline outperforms the pure baseline in all five scenarios and is not lower than the pure, Memento-only, and Hope-only variants in four of the five scenarios. Averaged across the suite, mission success rises from 0.6595 to 0.6828 and overall effectiveness rises from 0.7417 to 0.7924. These summary values are taken from the rerun scene-out summary and cross-checked against the scene-level `full_result.json` files.

Table 6 therefore establishes the broad cross-scenario pattern before the manuscript turns to scene-by-scene views. Table 7 then reports the scenario-level results for the full pipeline and the pure baseline, together with the mission-success confidence intervals reported by the stored Monte Carlo summaries. The largest mission-success gain appears in the coastal joint-assault

Table 6: Cross-scenario summary for the selected leave-one-source-scenario-out evidence set.

Summary item	Pure LLM	Full
Scenario count	5	5
Average mission success	0.6595	0.6828
Average overall effectiveness	0.7417	0.7924
Full wins against Pure LLM	–	5/5
Full not lower than Pure/Memento/Hope	–	4/5

scenario (+0.0515), while the smallest gain appears in the urban-defense scenario (+0.0104). Even when the mission-success gain is modest, the full system still improves overall effectiveness in every selected scenario in the refreshed evidence set.

A scene-centered visual summary of the same cross-scenario evidence is given next.

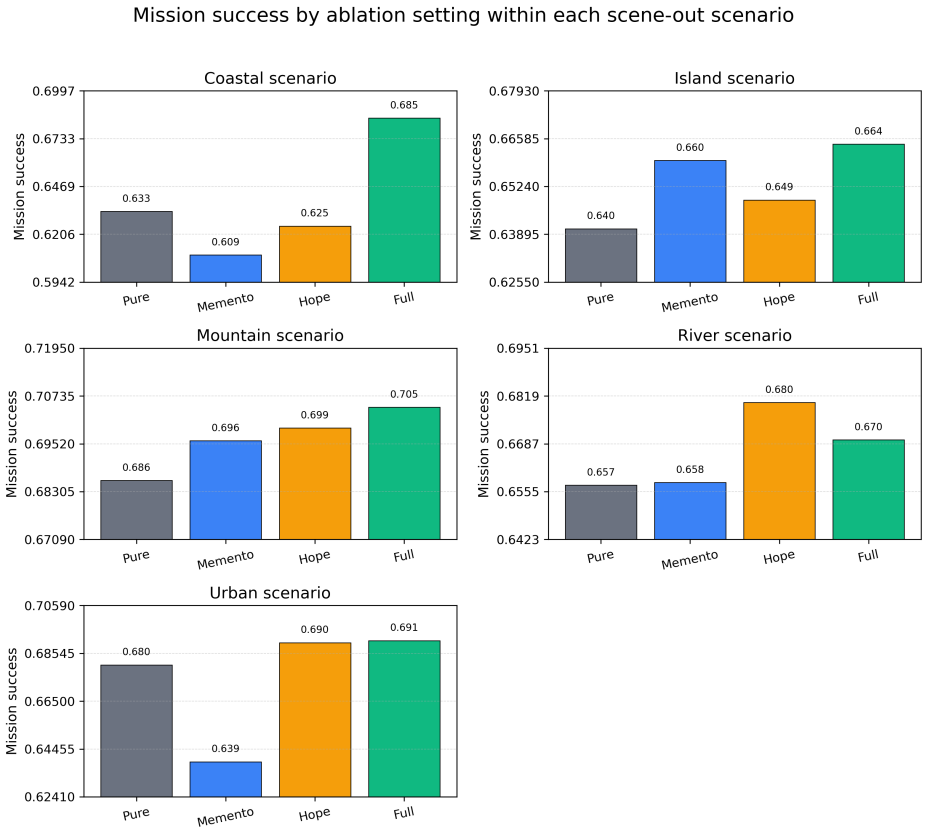

Figure 3: Mission success by ablation setting within each of the five scene-out scenarios in the canonical generalization suite. Each panel uses a locally tightened y-axis range to make between-group differences easier to inspect. The plotted values come from the corresponding `best_simulation.mission_success` fields in the per-scene `full_result.json` files.

Figure 3 visualizes the same cross-scenario evidence from a scene-centered angle. Each panel corresponds to one scenario and compares the four ablation settings directly. The Full pipeline remains the highest bar in four of the five panels and narrowly trails the Hope-only variant in the river-crossing case, whereas the relative ordering of the Memento-only and Hope-only variants changes strongly with scenario family.

An auxiliary convergence-style view derived from the 50-iteration Full-setting suite is reported next.

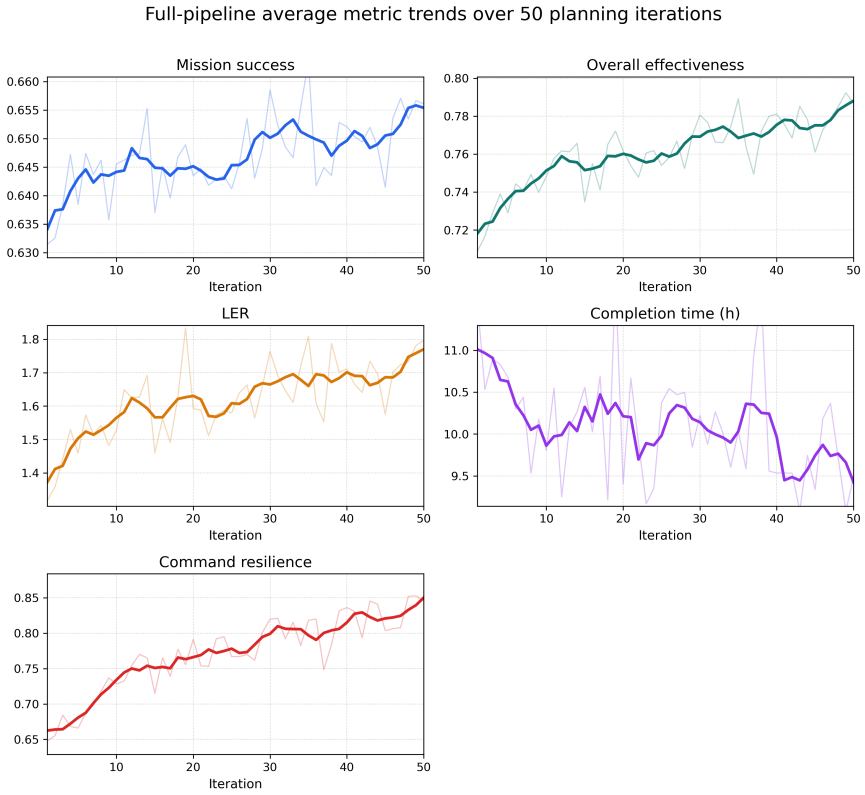

Figure 4: Average Full-pipeline metric trajectories over 50 planning iterations. Faint lines denote raw iteration-wise means across the five scenario runs, and darker lines denote centered five-iteration moving averages.

Figure 4 averages the Full-pipeline metric values over the five scenario families at each iteration index and therefore should not be mixed numerically with the 20-iteration tables above. It is included to show the longer-run tendency of mission success, overall effectiveness, LER, completion time, and command resilience more clearly.

Table 7 then reports the same cross-scenario evidence in row-wise form for the full pipeline and the pure baseline, together with the mission-success confidence intervals reported by the stored Monte Carlo summaries. The largest mission-success gain appears in the coastal joint-assault scenario (+0.0515), while the smallest gain appears in the urban-defense scenario (+0.0104). Even when the mission-success gain is modest, the full system still improves overall effectiveness in every selected scenario in the refreshed evidence set.

Table 7: Scenario-level generalization results for the selected evidence set.

Scenario	Family	Terrain	Pure MS	Full MS	Full − Pure MS	Full MS 95% CI
Coastal joint assault	assault	coastal	0.6332	0.6847	+0.0515	[0.6840, 0.6855]
Island resupply corridor	sustain	maritime	0.6405	0.6643	+0.0238	[0.6636, 0.6649]
Mountain corridor reconnaissance	recon	mountain	0.6859	0.7045	+0.0186	[0.7035, 0.7054]
River crossing breakthrough	assault	river	0.6573	0.6698	+0.0125	[0.6691, 0.6705]
Urban hub defense	defense	urban	0.6805	0.6909	+0.0104	[0.6901, 0.6918]

4.3. Interpretation

Two patterns are consistent across the refreshed evidence. First, adaptive weighting appears to be an important contributor, but not in a uniformly additive way. The Hope-only variant already improves several metrics substantially in the single-scenario ablation, and in some held-out scenes it remains competitive with or even slightly ahead of the Full setting on mission success. The stage-level view in Table 5 and Figure 2 makes that contribution more specific. The full pipeline does not raise all kill-chain stages uniformly. Instead, it shows its clearest recovery in `control` and `sustain`, while `breach` remains the persistent bottleneck and even slips slightly below the pure baseline in the selected sample scenario. This pattern is consistent with the feedback-update logic described in the method section: the controller reacts to stage deficits and reallocates capability emphasis, but the stored result traces suggest that repeated reweighting is better at reinforcing already responsive stages than at fully removing the hardest breach-stage weakness in the current sample scenario.

Second, memory support becomes more valuable when the case-bank construction is broadened, but its benefit remains uneven across scene families. The frozen seed bank used in the single-scenario ablation contains only five records, whereas the selected generalization evidence uses scene-out banks derived from 250 source cases. This broader pool plausibly contributes to the positive memory contribution in island-resupply and mountain-reconnaissance scenes, but the refreshed results also show clear negative transfer in coastal and urban cases. The current draft therefore treats memory-scale and memory-quality explanations as suggestive rather than proven.

At the same time, the results should be interpreted conservatively. The evidence supports improved simulation metrics inside the repository's evaluation framework; it does not, by itself, establish external validity, real-world operational value, or statistical significance beyond the stored Monte Carlo confidence summaries. No claim in this section should be read as an argument

that the selected evidence set exhausts all result directories in the repository or that unreported suites would necessarily follow the same pattern.

5. Discussion

The current manuscript draft is intentionally conservative. It treats the repository as the primary source of truth and avoids overstating what the present evidence can support.

First, the strongest aspect of the evidence is the repository's internal consistency. The same structured plan objects feed the generator, simulator, result serializer, and DoDAF/C2SIM exporters. This makes it possible to trace reported numbers back to stored JSON files and to inspect the logic that produced them. For a first journal draft, such traceability is more valuable than broader but weakly grounded claims.

Second, the selected evidence suggests that the interaction between memory augmentation, adaptive weighting, and reflection is useful within the current simulator, but also that the gains remain scenario-dependent. The full pipeline is best in the chosen single-scenario ablation and exceeds the pure baseline in all five scenarios of the refreshed cross-scenario suite, yet it does not dominate every intermediate ablation in every held-out scene. At the same time, the stage-level evidence shows that the gains are not uniform. As emphasized by the single-scenario stage table and stage-profile figure, `breach` remains the most persistent bottleneck and `disrupt` remains comparatively resistant to improvement even after iterative feedback updates. This matters because it links the discussion back to the method-level feedback logic: the controller does react to stage deficits and can reallocate capability emphasis, but the current evidence suggests that repeated reweighting is more effective at reinforcing responsive stages such as `control` and `sustain` than at fully removing the hardest breach-stage weakness. The practical consequence is that the paper should be read not only as a claim of average improvement, but also as a map of where the present pipeline still appears structurally fragile. Just as importantly, the present loop should not be mistaken for a full reinforcement-learning or search regime in the style of standard RL formulations, AlphaGo-scale tree search, or population-based training [47–49]. It is closer to a bounded local update rule operating over multiple partially competing mission metrics, a setting that shares some flavor with multi-task optimization trade-offs [50].

Several limitations remain before submission:

- **LLM backend traceability**: the refreshed canonical ablation and 20-iteration scene-out evidence now serialize machine-readable runtime manifests. The reported rerun manifests record `deepseek-r1-0528-qwen3-8b` in the Python virtual environment `Memento` on a 13th Gen Intel(R) Core(TM) i7-13700F CPU with an NVIDIA GeForce RTX 3060 GPU, using an OpenAI-compatible local endpoint. This removes the earlier dependence on author-confirmed runtime metadata for the main reported bundles.

- **Case-bank comparability**: the ablation evidence uses a five-record frozen seed bank, whereas the selected generalization evidence uses 250 source cases and 200-case scene-out banks. This is informative, but it also means that the role of memory quality and memory scale must be discussed carefully.

- **Figure scope**: the main-text figures are now generated directly from repository-local JSON files, and a separate graphical abstract has been prepared as a non-generative submission asset. The remaining work is narrower: final grayscale verification, any journal-specific artwork packaging, and any extra venue-driven figure refinements.

- **Metadata completeness**: the author list, affiliation, corresponding-author email, no-funding statement, competing-interest declaration, and CRediT roles are now recorded, but the final acknowledgments wording and data-availability wording may still need journal-specific refinement.

- **Build readiness**: the manuscript now compiles successfully with a workspace-local TinyTeX toolchain, but the final submission package still needs the remaining editorial metadata and any journal-specific packaging checks.

These limitations do not invalidate the draft; they define the next cleanup stage. The manuscript is already useful as a structured, source-traceable paper skeleton. The next iteration should focus on final grayscale and artwork checks, any remaining figure-only reruns needed for auxiliary plots, and the journal-specific submission package.

6. Conclusion

This paper draft documents a repository-implemented pipeline that combines case retrieval, adaptive capability weighting, reflection, structured simulation evaluation, and standards-oriented exporters for simulated joint-operation planning. Using a single canonical evidence scope, the draft reports that the Full Memento–Hope–Reflection configuration outperforms the pure LLM baseline in the selected ablation study and in all five scenarios of the selected cross-scenario generalization suite.

Just as importantly, the draft makes its current boundaries explicit. It does not claim formal proofs, external benchmark leadership, or evidence beyond what the repository can support. Instead, it establishes a clean Elsevier-style manuscript structure, a traceable evidence inventory, a figure set generated from repository-local JSON outputs, and a reproducible local PDF build. That gives the project a practical foundation for the next pass: polishing the final editorial metadata, refining the page layout, and preparing the journal-specific submission package.

CRediT authorship contribution statement

Changrui Zhang: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing - original draft. Chengwei Yang: Supervision, Validation, Resources, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research received no external funding. No additional non-funding acknowledgments are declared for this version of the manuscript.

Data availability

The materials that support this study are maintained in the authors' project repository. The corresponding author can provide the code, structured scenario files, archived result bundles, and manuscript assets on reasonable request, subject to any repository-release constraints that apply at the time of sharing.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Advances in Neural Information Processing Systems, 2017, pp. 5998–6008.

[2] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of NAACL-HLT, 2019, pp. 4171–4186.

[3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, D. Amodei, Language models are few-shot learners, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 1877–1901.

[4] OpenAI, GPT-4 technical report (2023). arXiv:2303.08774.
URL `https://arxiv.org/abs/2303.08774`

[5] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Roziere, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, G. Lample, LLaMA: Open and efficient foundation language models (2023). arXiv:2302.13971.
URL `https://arxiv.org/abs/2302.13971`

[6] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Costa, M.-A. Lachaux, G. Moreira, K. Mohammed, B. Roziere, A. Rodriguez, R. Taori, H. Martin, T. Wang, R. Stojnic, et al., Llama 2: Open foundation and fine-tuned chat models (2023). arXiv:2307.09288.
URL `https://arxiv.org/abs/2307.09288`

[7] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Chen, et al., A survey of large language models (2023). arXiv:2303.18223.
URL https://arxiv.org/abs/2303.18223

[8] J. Yang, H. Jin, R. Tang, X. Han, Q.-W. Feng, H. Jiang, B. Yin, X. Hu, J. Lu, H. Liu, et al., Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond, ACM Transactions on Knowledge Discovery from Data 18 (6) (2024) 1–32.

[9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems, Vol. 35, 2022, pp. 24824–24837.

[10] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, Proceedings of ICLR (2023).

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, Proceedings of ICLR (2023).

[12] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, K. Narasimhan, Tree of thoughts: Deliberate problem solving with large language models, Advances in Neural Information Processing Systems (2023).

[13] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, T. Scialom, Toolformer: Language models can teach themselves to use tools, Advances in Neural Information Processing Systems (2023).

[14] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[15] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, W.-t. Yih, Dense passage retrieval for open-domain question answering, in: Proceedings of EMNLP, 2020, pp. 6769–6781.

[16] J. Johnson, M. Douze, H. Jegou, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (3) (2019) 535–547.

[17] N. Reimers, I. Gurevych, Sentence-BERT: Sentence embeddings using siamese BERT-networks, in: Proceedings of EMNLP-IJCNLP, 2019, pp. 3982–3992.

[18] A. Graves, G. Wayne, I. Danihelka, Neural turing machines (2014). arXiv:1410.5401.
URL `https://arxiv.org/abs/1410.5401`

[19] J. Weston, S. Chopra, A. Bordes, Memory networks, Proceedings of ICLR (2015).

[20] S. Sukhbaatar, A. Szlam, J. Weston, R. Fergus, End-to-end memory networks, in: Advances in Neural Information Processing Systems, 2015, pp. 2440–2448.

[21] A. Aamodt, E. Plaza, Case-based reasoning: Foundational issues, methodological variations, and system approaches, AI Communications 7 (1) (1994) 39–59.

[22] J. Kolodner, Case-Based Reasoning, Morgan Kaufmann, San Mateo, 1993.

[23] N. Shinn, F. Cassano, E. Germanos, A. Gundersen, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, Advances in Neural Information Processing Systems (2023).

[24] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, M. Gupta, P. Clark, Y. Choi, H. Hajishirzi, A. Farhadi, A. Sabharwal, Self-refine: Iterative refinement with self-feedback, Advances in Neural Information Processing Systems (2023).

[25] Q. Guo, R. Wang, J. Guo, Y. Li, S. Liu, S. Diao, B. Zhang, M. Zhang, H. Ji, T. Wang, et al., Connecting large language models with evolutionary algorithms yields powerful prompt optimizers, Proceedings of ICLR (2024).

[26] H. Zhou, Y. Chen, S. Guo, et al., Memento: Fine-tuning LLM agents without fine-tuning LLMs (2025). arXiv:2508.16153.
URL https://arxiv.org/abs/2508.16153

[27] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein, Generative agents: Interactive simulacra of human behavior, in: Proceedings of UIST, 2023, pp. 1–22.

[28] Joint Chiefs of Staff, Joint Publication 3-60: Joint targeting, Washington, D.C.: Joint Chiefs of Staff (2013).

[29] J. A. Tirpak, Find, fix, track, target, engage, assess, Air Force Magazine 83 (7) (2000) 24–29.

[30] D. S. Alberts, J. J. Garstka, F. P. Stein, Network Centric Warfare: Developing and Leveraging Information Superiority, 2nd Edition, CCRP Publication Series, Washington, D.C., 1999.

[31] A. M. Law, Simulation Modeling and Analysis, 5th Edition, McGraw-Hill Education, New York, 2015.

[32] Department of Defense, DoD Architecture Framework Version 2.02, Washington, D.C.: Department of Defense (2010).

[33] IEEE Computer Society, IEEE 1516-2010: IEEE standard for modeling and simulation high level architecture—framework and rules, New York: IEEE (2010).

[34] Simulation Interoperability Standards Organization, SISO-STD-019-2020: Standard for command and control systems-simulation systems interoperation, Orlando: SISO (2020).

[35] C. L. Blais, M. Dechand, M. Dembach, et al., The use of automated reasoning with the command and control system to simulation system interoperation standard, Simulation Innovation Workshop (2021).

[36] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Computation 9 (8) (1997) 1735–1780.

[37] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean, Outrageously large neural networks: The sparsely-gated mixture-of-experts layer, Proceedings of ICLR (2017).

[38] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, Z. Chen, GShard: Scaling giant models with conditional computation and automatic sharding, Proceedings of ICLR (2021).

[39] W. Fedus, B. Zoph, N. Shazeer, Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity, Journal of Machine Learning Research 23 (120) (2022) 1–39.

[40] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, et al., GLaM: Efficient scaling of language models with mixture-of-experts, in: Proceedings of ICML, 2022, pp. 5547–5569.

[41] A. Komatsuzaki, J. Puigcerver, J. Lee-Thorp, N. Ruiz, K. Murthy, K. Guu, A. Kumar, G. Gur-Ari, H. Schwenk, J. Dean, C. Raffel, Sparse upcycling: Training mixture-of-experts from dense checkpoints (2022). arXiv:2212.05055.
URL `https://arxiv.org/abs/2212.05055`

[42] J. Puigcerver, A. Roy, B. Mustafa, G. Ghiasi, O. Firat, S. Borgeaud, A. Kumar, H. Eichner, Y. Huang, G. Driessche, et al., From sparse to soft mixtures of experts, Proceedings of ICLR (2024).

[43] S. Doubov, N. Sardana, V. Chiley, Sparse upcycling: Inference inefficient finetuning, NeurIPS Workshop on Efficient Natural Language and Speech Processing (2024).

[44] E. He, A. Khattar, R. Prenger, et al., Upcycling large language models into mixture of experts (2024). arXiv:2410.07524.
URL `https://arxiv.org/abs/2410.07524`

[45] X. Mao, Stochastic Differential Equations and Applications, 2nd Edition, Woodhead Publishing, Cambridge, 2007.

[46] H. K. Khalil, Nonlinear Systems, 3rd Edition, Prentice Hall, Upper Saddle River, NJ, 2002.

[47] R. S. Sutton, A. G. Barto, Reinforcement Learning: An Introduction, 2nd Edition, MIT Press, Cambridge, MA, 2018.

[48] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al., Mastering the game of go with deep neural networks and tree search, Nature 529 (7587) (2016) 484–489.

[49] M. Jaderberg, V. Dalibard, S. Osindero, W. M. Czarnecki, J. Donahue, A. Razavi, O. Vinyals, T. Green, I. Dunning, K. Simonyan, et al., Population based training of neural networks (2017). arXiv:1711.09846.
URL `https://arxiv.org/abs/1711.09846`

[50] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, C. Finn, Gradient surgery for multi-task learning, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 5824–5836.